Acta Astronautica 235 (2025) 463–474

Contents lists available at ScienceDirect

Acta Astronautica

journal homepage: www.elsevier.com/locate/actaastro

Research paper

Motion planning of free-flying space robots for tracking relative motion tumbling targets by sequential optimization via reinforcement learning

Zhicai Dong, Lu Wang *, Yiman Zhu, Liaoxue Liu, Jian Guo, Yu Guo

School of Automation, Nanjing University of Science and Technology, Nanjing, 210094, China

ARTICLE INFO

Keywords:
Space robots
Motion planning
Reinforcement learning

ABSTRACT

Uncontrolled satellites and space debris present significant challenges to space robot capture, particularly when these non-cooperative space objects (NCSOs) are generally tumbling and moving relative to the space robot. This requires space robots to actively adapt to target motion. Additionally, free-flying space robots (FFSRs) possess 9-DOF, resulting in an extensive exploration space that renders reinforcement learning-based tracking of complex tumbling targets impractical. To this end, we propose a novel Sequential Optimization Path Tracking (SOPT) algorithm, in which the FFSR is decomposed into two agents, thus the complexity of high-dimensional planning is significantly reduced. As a result, rapid and precise tracking of tumbling targets with relative motion can be achieved by the FFSR. Furthermore, we designed the joint tracking task framework that integrates multiple tracking subtasks, bridging various stages of the capture process. Simulation results validate the effectiveness and robustness of the proposed algorithm.

1. Introduction

The growing number of satellites launched into space for scientific and commercial purposes has led to many transitioning into non-cooperative space objects (NCSOs) as a result of on-orbit failures or fuel depletion. These NCSOs are unable to provide cooperative information such as velocity, position, or attitude, thereby posing potential threats to spacecraft and space operations. Space robots with on-orbit servicing capabilities hold great potential for the active capture and removal of NCSOs [1,2], which are uncontrolled and even tumbling erratically [3]. Furthermore, relative motion always exists between the space robot and the NCSO prior to capture, encompassing both translational and rotational components. There is only a limited time window available for capture. This requires the space robot to actively adapt to the NCSO for capture maneuvers, which poses a significant challenge to the motion planning of space robots.

Space robots can be categorized into three types based on the operational conditions of the base during contact operations [4]. Free-floating space robots operate with an uncontrolled base, which minimizes propellant consumption. However, the free-floating base severely restricts the capture range, requires significant consumption of non-renewable fuel to maintain the relative position with the target prior to capture. Space robots with pose-controllable base are capable of wide-area capture, but frequent maneuvers during capture consume large amounts of propellant, which significantly reduces on-orbit time [5]. Space robots with attitude-controllable base, stabilized by reaction wheels, require only a small amount of renewable energy for base attitude control. This configuration effectively suppresses vibrations caused by manipulator motions and provides a significantly larger capture range compared to floating-base space robots. Taking into account practical factors such as energy consumption and capture range, the space robot with attitude-controllable base is better suited for capturing NCSOs that are tumbling and moving relative to the space robot during the approach phase. Unless otherwise specified, the free-flying space robot (FFSR) in this paper refers to a space robot with attitude-controllable base.

However, the planning of motions for FFSRs is a challenging undertaking, primarily due to the highly coupled and nonlinear dynamics inherent to these systems. The motion of the manipulator affects the base motion, which in turn affects the manipulator motion. Although model-based controllers have been used for coupling control of such simplified nonlinear systems, the traditional approach of integrating a motion planner with an inverse kinematics solver must address issues such as the singularity of the generalized Jacobian matrix and the difficulty of parameter identification [6]. Furthermore, as the degrees of freedom of robots increase, their modeling and control become much difficult and complex [7].

For the above issues, model-free methods have garnered increasing attention from researchers. Model-free methods do not rely on kinematic or dynamic models of the FFSRs. Researchers derive optimal

* Corresponding author.
E-mail addresses: dongzhicai@njust.edu.cn (Z. Dong), wanglu21@njust.edu.cn (L. Wang).

https://doi.org/10.1016/j.actaastro.2025.05.027
Received 17 January 2025; Received in revised form 1 April 2025; Accepted 15 May 2025
Available online 8 June 2025
0094-5765/© 2025 IAA. Published by Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

controllers by iteratively tuning controller parameters using optimization algorithms [8]. Optimization-based trajectory planning methods perform well in static environments. Without calculating joint inverse kinematics, Ni [9] iteratively optimizes joint angles for obstacle avoidance and base attitude adjustment via the particle swarm optimization (PSO) algorithm. Yan [10] uses multi-objective particle swarm optimization (MOPSO) to optimize arm angles, minimizing base disturbance and maximizing manipulability during the approach phase. However, with the movement or tumbling of the target, the entire trajectory must be recalculated continuously, which fails to meet real-time requirements. These limitations hinder their practical implementation in space-borne systems.

Classical deep reinforcement learning (RL) methods have proven to be effective in addressing space robot planning challenges in specific scenarios, with the learned policy being resilient to external disturbances and adaptable to different environments [11]. Dong [12] employed a two-axis matching method to reduce the complexity of orientation exploration, enabling the free-floating space robot to successfully track a tumbling target. Hovell [13] successfully implemented a space robot operating in 2D plane to capture an NCSO and simultaneously stabilize it using the D4PG algorithm. However, to capture tumbling targets with relative motion based on RL methods, we still need to overcome some significant challenges. The first problem is that most of the research in the field has focused on the control of space manipulators rather than satellite bases [14–16], most of which concentrate only on manipulator position control [17–19]. Whereas coupled controllers that simultaneously control the base attitude can greatly extend the capture range of space manipulators, the resulting problems of slow or no convergence in high-dimensional planning problems have still not been effectively addressed. Another problem is that, although the current research has been able to achieve the tracking of relatively moving tumbling targets in 2D plane [5,13], the tracking of NCSOs by the space robot in 3D space is still based on the assumption that space robot and NCSO are relatively stationary. In reality, there is always a relative motion between the space robot and the NCSO prior to capture, which means that the space robot actively adapts to the NCSO motion for capture maneuver is still a task to be accomplished.

Furthermore, there are usually two phases of tasks that need to be completed prior to the actual capture (physical contact). The observation and planning phase requires obtaining information about the motion and physical characteristics of the NCSO. The information about the NCSO can only be measured by the sensors of the FFSR. This implies that the FFSR must actively track the NCSO visually to keep it within the field of view (FOV) of its sensors [20]. The objective of the FFSR in the second phase is to align with the pose of the target precisely. However, in previous work, separate trajectory controllers have generally been designed for active visual tracking and target pose tracking, treating them as independent tasks [21,22]. In fact, the two tasks are closely related and must be performed sequentially. In addition, different tasks, space robot structures, and environmental settings lead to poor controller generalization, as the adaptation of single-task trajectory controllers to joint tasks is not feasible. Therefore, existing methods are not yet able to concatenate the multi-task process of capturing an NCSO.

To address the above problems, the Sequential Optimization Path Tracking (SOPT) algorithm is proposed for FFSRs. The SOPT framework decouples the base and the manipulator as two agents for sequential optimization to reduce the complexity of the optimization. The manipulator active visual tracking, target tracking, base reorientation, and base attitude stabilization tasks are integrated into a joint tracking task. The obtained policy allows the FFSR to approach and track tumbling targets in relative motion passing through the reachable workspace. The main contributions are as follows:

- We propose the Sequential Optimization Path Tracking algorithm to decouple the manipulator and the base, which greatly reduces the complexity of learning.
- We construct the joint tracking task framework to concatenate the multi-task flow prior to capture, which makes the algorithm practical.
- The proposed algorithm reduces the limitations on the motion state of the captured target. In addition, tracking relatively moving targets broadens the operational scenarios of space robots, benefiting other on-orbit service tasks.

2. Problem statement

In this section, we present the preliminary knowledge of the task of tracking a tumbling target with relative motion in our work, including the structure of the space robot, motion characteristics of NCSOs, and the simulation environment. It is worth noting that our model-free approach does not rely on the kinematic and dynamic models of FFSR or NCSOs, which will not be further elaborated in this section.

2.1. Structure of the space robots

Free-floating space robots are favored for their fuel efficiency, but their capture missions typically assume that the target remains relatively stationary within the dexterous workspace, requiring significant fuel consumption for pre-mission maneuvers. If the space robot can directly track and capture targets in relative motion at close range without the need for multiple maneuvers to adjust the base position, this will greatly extend the on-orbit lifetime. Therefore, the tracking task in this study chooses the FFSR with only attitude-controllable base as the controlled plant. The robot comprises an n-DOF serial manipulator and an attitude-controllable base, with a specific focus on a 6-DOF manipulator mounted on a cubic base. It is worth noting that in our work we are only concerned with the pre-capture task and therefore ignore the gripper shape.

2.2. Analysis of NCSOs motion

NCSOs typically orbit the Earth in an elliptical orbit in a tumbling state. Their motion state consists of translation and tumbling. The direction of the translation velocity changes continuously on large spatial scales. In small-scale, close-range scenarios, the motion of an NCSO can be approximated as linear motion with a constant velocity. Practical NCSOs generally exhibit tumbling motion in the forms of rotation, precession, and nutation. The complexity of their motion arises from their nonlinear characteristics and multi-dimensional degrees of freedom. In the absence of external forces, when the initial angular velocity vector forms an angle θ with the inertia axis, the NCSO undergoes conical motion of the spin axis around the angular momentum axis $\boldsymbol{H}$. To reflect the commonality of the shape of the NCSO, we simplify the target as a cube, as shown in Fig. 1.

2.3. Simulation environment

Due to the difficulty of reproducing the space environment on the ground, the experiments were performed in the dynamics simulation environment MuJoCo. To focus on the application feasibility of the method, a 6-DOF UR5 robot was chosen as the manipulator, using its official kinematic and dynamic parameters. Cameras are mounted on the end-effector of the manipulator and around the base. Define the set of points that can be reached by the manipulator through any orientation in the fixed base attitude as the dexterous workspace [23]. Define the set of theoretically reachable points of the manipulator with a controlled base attitude as the reachable workspace [24].

In this work, we consider a more realistic mission scenario, where the target still has a velocity difference with the space robot after long-range maneuvering and the target does not pass through the

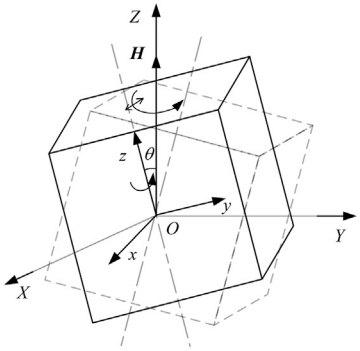


Fig. 1. The tumbling motion of the NCSO.

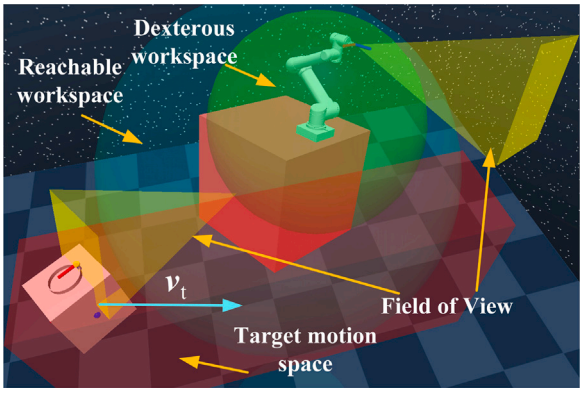


Fig. 2. Virtual task scenario in the MuJoCo simulator.

dexterous workspace, but only intersects the reachable workspace with its trajectory. To avoid fuel consumption from positional maneuvers, the space robot achieves the task via base attitude adjustment and manipulator control. In the simulation, the tumbling target will move along a line with uniform velocity $\boldsymbol{v}_t$ in any direction in the target motion space, as shown in Fig. 2.

3. Sequential optimization path tracking

To capture relative motion NCSOs, the SOPT algorithm is proposed to guide the space robot to track the tumbling target whose motion trajectory passes through the reachable workspace. The algorithm framework contains the joint task level and the sequential optimization level. The joint task level integrates the base reorientation, base stabilization, manipulator active visual tracking and target tracking tasks. The sequential optimization level decouples the base and the manipulator as two agents for optimization. Fig. 3 illustrates the framework for sequential optimization path tracking algorithm.

3.1. Joint task level

3.1.1. Base reorientation

Due to the initial trajectory of the target only passing through the reachable workspace, the base attitude needs to be adjusted to incorporate the target trajectory into the dexterous workspace to obtain a limited capture time window, i.e., the line segment $\|\overrightarrow{P_1P_2}\|$ where the trajectory intersects the dexterous workspace, as shown in Fig. 4. The attitude of the base is expressed as $\boldsymbol{R}_b = [\boldsymbol{x}_b, \boldsymbol{y}_b, \boldsymbol{z}_b]$. The central position of the dexterous workspace is denoted by $\boldsymbol{P}_d$. $\boldsymbol{H}_d$ is the vertical foot from $\boldsymbol{P}_d$ to the line where $\boldsymbol{v}_t$ is located. The initial pose of the base is highlighted in red.

There exists innumerable base attitudes that can incorporate the target motion trajectory into the dexterous workspace, and thus no definitive base attitude can be given. We transform the base reorientation task into an optimization problem by generating the base attitude online in real time through the RL method.

The capture time window length is one of the most important factors in measuring the degree of manipulability. Since $\|\overrightarrow{P_dH_d}\|$ is negatively correlated with $\|\overrightarrow{P_1P_2}\|$, we choose it to reflect the capture time window length, denoted as:

$$L_{ctw} = \max(\|\overrightarrow{P_dH_d}\| - U_{ctw}, 0), \tag{1}$$

where U_{ctw} is the capture time window length tolerance.

Although the dexterity workspace is theoretically approximated as a sphere, in fact it is only 3/4 spherical due to base shading. In addition, the base must not coincide with the target's trajectory. These implies that the base needs to be selected with a suitable working surface to ensure manipulability. Therefore, the angle φ_{ws} between $\boldsymbol{y}_b$ and $\boldsymbol{v}_t$ is calculated as the working surface metric by:

$$\varphi_{ws} = \max\left(\operatorname{acos}\left(\frac{\boldsymbol{y}_b \cdot \boldsymbol{v}_t}{\|\boldsymbol{y}_b\| \, \|\boldsymbol{v}_t\|}\right) - U_{ws}, 0\right), \tag{2}$$

where U_{ws} is the working surface tolerance.

The fuel consumption from the base attitude adjustment will determine its operating life. The amount of change in the current attitude $\boldsymbol{R}_b$ from the initial attitude $\boldsymbol{R}_{b_init}$ is used as the energy consumption metric, denoted as:

$$E_{cost} = \operatorname{acos}\left(\frac{\operatorname{tr}(\boldsymbol{R}_b \boldsymbol{R}_{b_init}^{-1}) - 1}{2}\right). \tag{3}$$

Considering the above metrics, the objective function for the base reorientation optimization task is established as:

$$B_{reori} = w_{ctw} L_{ctw}^2 + w_{ws} \varphi_{ws}^2 + w_{cost} E_{cost}, \tag{4}$$

where w_{ctw}, w_{ws} and w_{cost} are non-negative coefficients.

3.1.2. Base stabilization

Due to dynamic coupling, keeping the base attitude stable can improve the accuracy of the manipulator operation. It is worth noting that theoretically it is possible to complete the base task when the target is far away and complete the manipulator task with the stable base. However, the further away the target is, the larger the measurement error of the motion state. There is a risk of multiple base attitude adjustments. Furthermore, to verify the potential operation capability of space robots, we consider more difficult task scenarios, i.e., moving base-moving manipulator scenarios. The base and manipulator tasks are performed simultaneously, with a short capture time window. The manipulator performs active visual tracking during the dynamic adjustment of the base attitude. However, the base should have entered the stabilization phase by the time the manipulator tracks the target pose, i.e. there is a time constraint on the base task switching. The base stabilization is measured by the base angular velocity $\boldsymbol{w}_b$, and the base stabilization metrics with time constraints $U_{t_{reori}}$ is established as:

$$B_{stab} = \begin{cases} 0, & \text{if } t_{reori} \le U_{t_{reori}} \\ w_{stab}\|\boldsymbol{w}_b\|, & \text{else} \end{cases}, \tag{5}$$

where w_{stab} is a non-negative coefficient, and t_{reori} denotes the completion time of the base reorientation.

3.1.3. Active visual tracking

For active vision tracking tasks, many researchers have focused their work on target pose estimation algorithms, assuming space robots as arbitrarily movable objects in 3D space, ignoring the limitations of the camera carrier [25]. In fact, space robots maneuver via propellant only for essential tasks, and the base FOV is almost fixed. Therefore, only the camera on the manipulator can be used to actively acquire NCSO information for the capture phase [26].

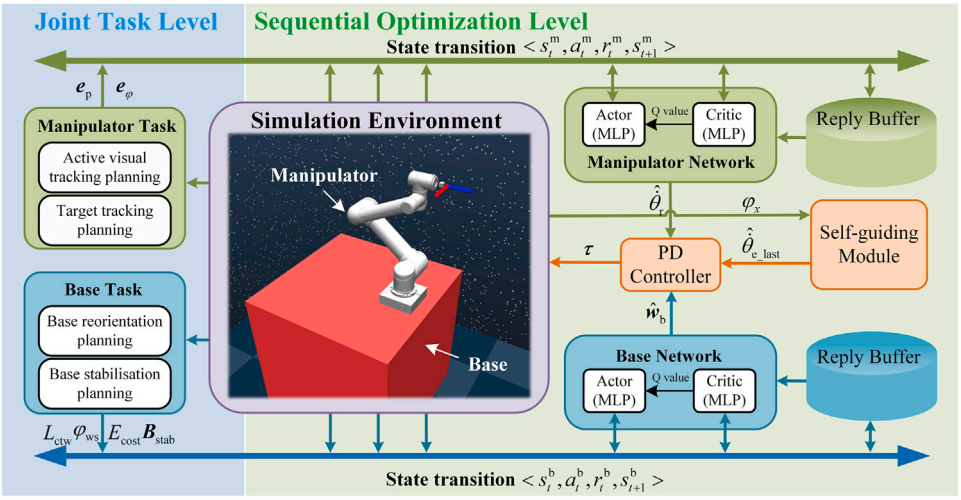


Fig. 3. Framework of SOPT algorithm. The joint task level will be introduced in Section 3.1. The sequential optimization level is described in detail in Section 3.2.

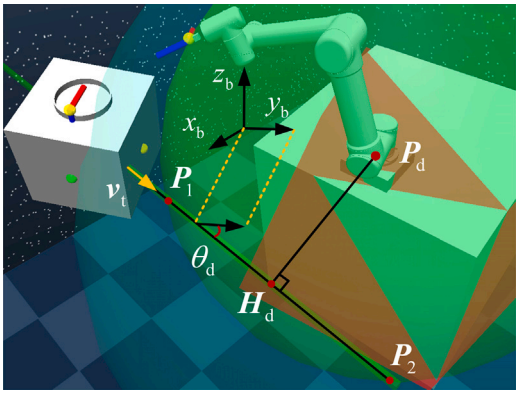


Fig. 4. Motion planning design for base reorientation.

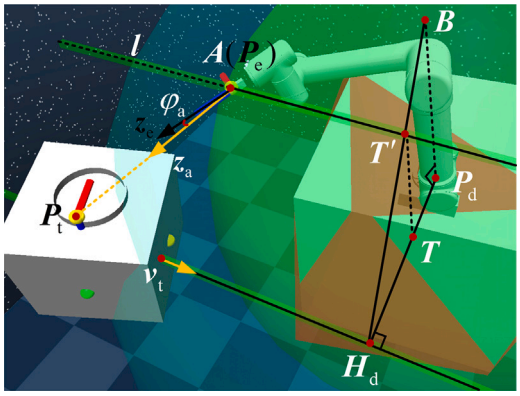


Fig. 5. Motion planning design for active visual tracking.

In essence, active vision tracking and target position tracking are all about controlling the manipulator to reach the desired pose, only the purpose of the task is different. Similar to base reorientation, there are innumerable poses in which the manipulator can perform target visual tracking. The difference is that we design a special active visual tracking rule to obtain the determined desired pose by taking into account the capture time window and task articulation, as shown in Fig. 5. The motion planning based on RL has the problem of huge attitude state space. Referring to [12], two-axis angle is used to represent the attitude constraints involved in the task to improve the efficiency of the algorithm.

To ensure the target remains within the field of view of the end-effector camera and allows the sensors to provide more estimation data, the desired attitude should be orientated towards the target. The line connecting the target position $\boldsymbol{P}_t$ and the current end-effector position $\boldsymbol{P}_e$ is defined as the desired attitude axis $\boldsymbol{z}_a$, denoted as:

$$\boldsymbol{z}_a = \frac{\overrightarrow{\boldsymbol{P}_t\boldsymbol{P}_e}}{\|\overrightarrow{\boldsymbol{P}_t\boldsymbol{P}_e}\|}. \tag{6}$$

The target is guaranteed to be in the FOV by the coincidence of the $\boldsymbol{z}_e$ axis of the manipulator with $\boldsymbol{z}_a$. By rotating the angle φ_a, the vector of $\boldsymbol{z}_e$ can be adjusted to $\boldsymbol{z}_a$, determined by the following equation:

$$\varphi_a = \operatorname{acos}\left(\frac{\boldsymbol{z}_e \cdot \boldsymbol{z}_a}{\|\boldsymbol{z}_e\| \, \|\boldsymbol{z}_a\|}\right). \tag{7}$$

To articulate the two tasks of the manipulator, the end-effector position should be progressively closer to the tumbling target as the base attitude adjusting. $\boldsymbol{T}$ on $\overrightarrow{\boldsymbol{P}_d\boldsymbol{H}_d}$ is progressively closer to the $\boldsymbol{H}_d$ as the target progressively approaches the dexterous workspace, denoted as:

$$\boldsymbol{T} = w_{ap1}\tanh(w_{ap2} - w_{ap3}\|\overrightarrow{\boldsymbol{P}_t\boldsymbol{H}_d}\|) + w_{ap4}, \tag{8}$$

where $w_{ap1}, w_{ap2}, w_{ap3}$ and w_{ap4} are non-negative coefficients.

It is worth noting that there is a risk of collision with the base. For task safety, the area around the tracking path should be kept as clear as possible. Setting $\overrightarrow{\boldsymbol{B}\boldsymbol{P}_d} \perp \overrightarrow{\boldsymbol{P}_d\boldsymbol{H}_d}$, it can project $\boldsymbol{T}$ onto $\overrightarrow{\boldsymbol{B}\boldsymbol{H}_d}$ to obtain $\boldsymbol{T}'$, which is obtained by the following equation:

$$\boldsymbol{T}' = \boldsymbol{H}_d - \frac{\|\overrightarrow{\boldsymbol{T}\boldsymbol{H}_d}\|}{\|\overrightarrow{\boldsymbol{P}_d\boldsymbol{H}_d}\|}\overrightarrow{\boldsymbol{B}\boldsymbol{H}_d}. \tag{9}$$

To maximize the capture time window, a straight line $\boldsymbol{l}$ along $\boldsymbol{T}'$ parallel to $\boldsymbol{v}_t$ is denoted as:

$$\boldsymbol{l}(t_a) = \boldsymbol{T}' + t_a\frac{\boldsymbol{v}_t}{\|\boldsymbol{v}_t\|}, \tag{10}$$

where t_a is the parameter variable. The boundary of the dexterous workspace is expressed as:

$$(\boldsymbol{x} - \boldsymbol{P}_d)^2 = r^2, \tag{11}$$

where r is the dexterous workspace radius, and $\boldsymbol{x}$ is a point on the boundary of the dexterous workspace. Substitute Eq. (10) for $\boldsymbol{x}$ in Eq. (11). Solve the quadratic equation to obtain two points of intersection of the line $\boldsymbol{l}$ with boundary of the dexterity workspace. The point closer to the target is selected as the desired position $\boldsymbol{A}$. Thus,

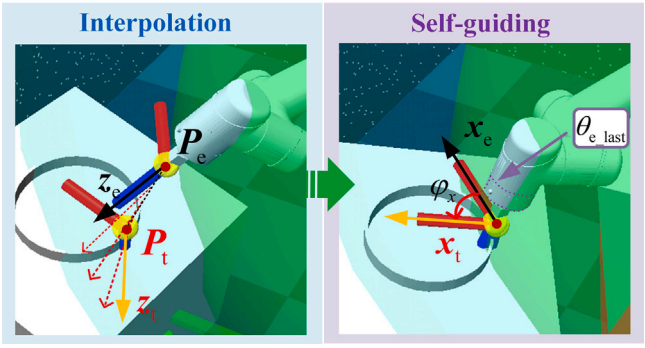


Fig. 6. Motion planning design for target tracking.

the desired pose of the manipulator with active visual tracking can be obtained.

3.1.4. Target tracking

When the end-effector is very close to the target, we need to switch the desired visual tracking pose to the target tracking pose and ensure that the trajectory is smooth and continuous, as shown in Fig. 6.

Linear interpolation is used to connect the current end-effector position $\boldsymbol{P}_e$ to the target position $\boldsymbol{P}_t$, denoted as Eq. (12), where t_{line} and $\boldsymbol{P}_{e_line}$ are the linear interpolation parameter and the end-effector desired position, respectively:

$$\boldsymbol{P}_{e_line} = (1 - t_{line})\boldsymbol{P}_e + t_{line}\boldsymbol{P}_t, \tag{12}$$

where t_{line} is the linear interpolation parameter.

To avoid the non-unitarisation problem caused by linear interpolation, spherical linear interpolation is used to interpolate the current end-effector attitude axis $\boldsymbol{z}_e$ to the target attitude axis $\boldsymbol{z}_t$, denoted as:

$$\theta_{sl} = \operatorname{acos}(\boldsymbol{z}_e \cdot \boldsymbol{z}_t), \tag{13}$$

$$\boldsymbol{z}_{e_sl} = \frac{\sin[(1 - t_{sl})\theta_{sl}]}{\sin(\theta_{sl})}\boldsymbol{z}_e + \frac{\sin(t_{sl}\theta_{sl})}{\sin(\theta_{sl})}\boldsymbol{z}_t, \tag{14}$$

where t_{sl} and $\boldsymbol{z}_{e_sl}$ are the spherical linear interpolation parameter and the end-effector desired attitude axis, respectively.

When the end-effector pose meets the tolerance of the target pose, the final alignment operation is performed using the self-guiding module from the two-axis matching method. The angular velocity $\dot{\theta}_{e_last}$ of the last joint is independently controlled using Eq. (15). Therefore, $\dot{\theta}_{e_last}$ does not need to be included in the action space.

$$\hat{\dot{\theta}}_{e_last} = w_\varphi \varphi_x, \tag{15}$$

where w_φ is a non-negative coefficient, and φ_x represents the angle between the target x-axis $\boldsymbol{x}_t$ and the end-effector x-axis $\boldsymbol{x}_e$.

After completing the interpolation, the desired pose of the end-effector is the target pose. It is worth noting that the target tracking task in this article refers to the end-effector's continuous stable and accurate pose synchronization under the target's translational and tumbling states during the capture time window, which in turn provides the necessary conditions for the subsequent capture task. Specifically, the pose error between the end-effector and the target during the time when the target is in the dexterous workspace must always be within the pose tolerance.

3.2. Sequential optimization level

In the sequential optimization level, we need to keep tracking the desired pose generated in the joint task level under multiple constraints. Therefore, the above trajectory planning task can be formulated as an optimization problem based on RL as follows:

$$\max_{\pi} \mathcal{J}(\pi) = \sum_{t=0}^{T} \mathbb{E}_{\pi}\left[\gamma^t R(\boldsymbol{s}_t, \boldsymbol{a}_t)\right]$$
$$\text{s.t.} \begin{cases} \| * \| \le U_*, * \in \left[e_p, e_\varphi, \theta_i, \dot{\theta}_i, \boldsymbol{w}_b, t_{reori}\right] \\ \{\lambda \boldsymbol{v}_t \mid \lambda \in \mathbb{R}\} \cap \mathcal{W} \neq \varnothing \\ \mathcal{M} = \varnothing \end{cases}, \tag{16}$$

where U_* denotes the task tolerance corresponding to $*$. e_p and e_φ denote the position error and attitude error, respectively. $\mathcal{M}$ and $\mathcal{W}$ denote collision space and dexterous workspace, respectively.

3.2.1. Sequential optimization training

Inspired by event-based alternating optimization (EAO) algorithms [27] and training methods in adversarial environments with fixed opponent policies [28], we proposed a sequential optimization training algorithm to facilitate the search for optimal policies by two agents with serial dependencies, effectively reducing the complexity of high-dimensional planning.

The structural characteristics of space robots offer possibilities for sequential optimization. The manipulability of the manipulator relies on the base attitude, but the base attitude is only slightly affected by the motion from the manipulator. The base is first trained to ensure that the target enters the dexterous workspace, and then the manipulator is trained while maintaining the current base policy. Since a deterministic policy gradient algorithm is used to train the base, the training environment for the manipulator remains stationary after the base deploys its deterministic policy, thus avoiding the issue of environmental non-stationarity. In addition, the self-guiding module can replace the search of the last manipulator joint. Thus SOPT transforms the 9-dimensional planning problem into a 3- and 5-dimensional problem, and the significant reduction in task complexity makes learning possible.

3.2.2. Backbone algorithm

We choose the Deep Deterministic Policy Gradient (DDPG) algorithm [29] and the Soft Actor-Critic (SAC) algorithm [30] as backbone methods to separately train the base and the manipulator. Both are off-policy algorithms that use a replay buffer to store samples, reducing sample correlation, and both adopt the actor-critic architecture.

Despite many similarities, the two algorithms differ significantly in terms of policy updating. DDPG updates the policy by combining the efficiency of deterministic policy gradient and Q-learning. In DDPG, the policy is updated by maximizing the Q-value provided by the Critic network, as follows:

$$\nabla_{\theta_D} J(\pi) = \mathbb{E}_{s\sim D}\left[\nabla_a Q_{\phi_D}(s,a)\nabla_{\theta_D}\pi_{\theta_D}(s)\right]. \tag{17}$$

The Critic network of DDPG updates the Q-function by minimizing the Bellman error:

$$L(\phi_D) = \mathbb{E}_{(s,a,r,s')\sim D}\left[\left(Q_{\phi_D}(s,a) - y\right)^2\right], \tag{18}$$

where $y = r + \gamma Q_{\phi'_D}(s', \pi_{\theta_D}(s'))$.

Unlike DDPG, SAC uses stochastic policies and encourages diversity of policy by introducing an entropy term, which motivates the agent to explore more.

SAC policy networks are updated by minimizing Kullback–Leibler (KL) discretization:

$$J_\pi(\theta_S) = \mathbb{E}_{s\sim D}\left[\min_{i=1,2} Q_{\phi_{Si}}(s,a) - \alpha \log \pi_{\theta_S}(a|s)\right]. \tag{19}$$

SAC mitigates the problem of overestimation of Q-value by using a dual Q-network structure. The parameters of q are trained by minimizing the Bellman residual, defined as:

$$J_Q(\phi_S) = \mathbb{E}_{(s,a,r,s')\sim D}\left[\left(Q_{\phi_{Si}}(s,a) - y\right)^2\right], \tag{20}$$

where $y = r + \gamma \left[\min_{i=1,2} Q_{\phi'_{Si}}(s', a') - \alpha \log \pi_{\theta_S}(a'|s')\right]$.

In both SAC and DDPG, the target actor and the critic network are updated by the soft update to ensure the stability of the training process:

$$\theta' \leftarrow \chi\theta + (1-\chi)\theta', \phi' \leftarrow \chi\phi + (1-\chi)\phi'. \tag{21}$$

For the pseudo code of the algorithm proposed in this paper, see Algorithm 1.

Algorithm 1 Sequential Optimization Path Tracking (SOPT)

Initialize $\theta_D, \theta'_D, \phi_D, \phi'_D, \theta_S, \theta'_S, \phi_{S1}, \phi'_{S1}, \phi_{S2}, \phi'_{S2}$
for $k = 1, 2$ **do**
  Initialize replay buffer $\mathcal{D}^k$
  $k = 2$: Freeze $\theta_D, \theta'_D, \phi_D, \phi'_D$
  for episode $m = 1, M_k$ **do**
    for step $t = 1, T-1$ **do**
      $k = 1$: Calculate base task metrics B_{reori} and B_{stab}.
      $k = 2$: Calculate the desired pose $\boldsymbol{P}_{\text{des}}, \boldsymbol{z}_{\text{des}}$ of the manipulator task.
      Sample a action a_t from $\pi_\theta(a_t|s_t)$
      Execute the action a_t and observe a new state s_{t+1}
      Store $< s_t, a_t, r_t, s_{t+1} >$ into $\mathcal{D}^k$
    end for
    for iteration $n = 1, N$ **do**
      Sample a minibatch from the replay buffer $\mathcal{D}^k$
      $k = 1$: Update the θ_D, ϕ_D via Eq. (17)-(18)
      $k = 2$: Update the $\theta_S, \phi_{S1}, \phi_{S2}$ via Eq. (19)-(20)
      Update the parameters of target networks via Eq. (21)
    end for
  end for
end for

3.2.3. Training details

The states of the space robots are all observable data, which can be obtained by their own on-board sensors. Since the robotic joint information does not directly reflect the impact on the base, only the 3D force f information of the robotic platform on the base is included in the base state space. The state space of the base is

$$s_t^b = \langle \boldsymbol{q}_b, \boldsymbol{w}_b, \boldsymbol{v}_t, L_{ctw}, \varphi_{ws}, E_{cost}, B_{stab}, f \rangle, \tag{22}$$

where $\boldsymbol{q}_b$ is the base attitude angle. The action space a_t^b is defined as the base angular velocity.

The angle θ_{e_last} of the last joint is controlled by the self-guiding module. As a result, this joint information is excluded from the manipulator state space. The manipulator state space is denoted as

$$s_t^m = \left\langle \boldsymbol{\theta}_r, \dot{\boldsymbol{\theta}}_r, \boldsymbol{P}_e, \boldsymbol{P}_{\text{des}}, \boldsymbol{z}_e, \boldsymbol{z}_{\text{des}}, \boldsymbol{e}_p, \boldsymbol{e}_\varphi \right\rangle, \tag{23}$$

including the joint angle $\boldsymbol{\theta}_r \in R^5$, the joint angular velocity $\dot{\boldsymbol{\theta}}_r \in R^5$, all desired positions $\boldsymbol{P}_{\text{des}} \in R^3$ and desired axes $\boldsymbol{z}_{\text{des}} \in R^3$. To ensure smoother motion of the manipulator, the agent uses the joint angular velocity as the action space a_t^m, rather than relying on joint angles.

The controller consists of the policy network and the driver. The policy network outputs the desired base attitude velocity and joint angular velocity, and the network parameters are updated by RL algorithms. The driver is implemented by a PD controller with fixed parameters for velocity tracking. At each time step of the control process, each policy network maps current state information to desired velocities, and then the driver maps these desired velocities to joint torques. The torques are ultimately applied in the simulation to control the space robot. In addition, due to the dynamic coupling, the manipulator motion affects the base attitude, so we randomly set the manipulator motion to enable the base policy to overcome the above perturbations during the training of the base.

In reinforcement learning, the design of effective reward plays a key role in improving the performance of the policy. Since the desired attitude cannot be obtained directly by the base task, the objective function of the base reorientation task is set as the main reward to meet the manipulability requirement with low energy consumption. In addition, the time constraint in the base stabilization task is added to the reward function as a penalty. The following equation is the base reward function:

$$r_b = -w_{b1} B_{\text{reori}} - w_{b2} \tanh(w_{b3} B_{\text{reori}} - w_{b4}) + w_{b5} - B_{\text{stab}}, \tag{24}$$

where $w_{b1}, \ldots, w_{b5}$ are non-negative coefficients.

We obtained the desired pose of the end-effector through the trajectory planning of 3.1.3 C-D in series with active vision tracking and target pose tracking tasks. Thus the pose error is used as a reward constructor.

$$r_{e_*} = -w_{m1} e_* - w_{m2} \tanh(w_{m3} e_* - w_{m4}) + w_{m5}, \tag{25}$$

where $w_{m1}, \ldots, w_{m5}$ are non-negative coefficients, the error e_* denotes the position error e_p or orientation error e_φ.

To meet the capture precision requirements, we set strict tolerances of $U_{e_p} = 0.015$ m and $U_{e_\varphi} = 0.017$ rad. Therefore, we introduced an additional success reward, as defined in Eq. (26), to encourage the manipulator to successfully track the desired dynamic pose.

$$r_s = \begin{cases} 4, & \text{if } e_p \le U_{e_p}, e_\varphi \le U_{e_\varphi} \\ 0, & \text{else} \end{cases}. \tag{26}$$

It is worth noting that there is no separate collision penalty included in the reward. After a collision, the specially designed error reward r_{e_*} will produce large negative rewards, prompting the policy network to learn the collision avoidance policy. Finally, the manipulator reward function is:

$$r_m = r_{e_p} + r_{e_\varphi} + r_s. \tag{27}$$

4. Simulation results

The simulator was implemented on a computer equipped with an NVIDIA TITAN XP GPU and an Intel Core i7-13700K CPU. The joint task framework and environment details, and the algorithm hyperparameters are provided in Tables 3–5 in Appendix A.

In the trajectory planning task, the NCSO moves within a $0.6 \times 2.6 \times 1.2$ m^3 rectangle. The initial position is randomly set on the 0.6×1.2 m^2 side of the rectangle. The target velocity direction is randomized, with a translational velocity of 0.1 m/s and a tumbling speed of [0.1, 0.1, 1] rad/s. Base angular velocity $\boldsymbol{w}_b$ not higher than 0.13 rad/s. The velocity of each joint of the manipulator during base training is set randomly within $[-0.1, 0.1]$ rad/s. Training consists of 500 episodes for the base and 1000 episodes for the manipulator. Set the random seeds to 0, 100 and 300.

4.1. Performance comparison

To test the effectiveness of our method, we conducted a series of comparison experiments.

- **Single-agent centralized training**: Given the successful application of SAC and DDPG in various robotic control tasks, we redesigned the single-agent centralized training paradigm based on the task to verify the rationality of the agent division method. Specifically, the self-guiding module remains in use. We combine the state space, action space, and rewards of both the base and manipulator, reformulating the task as an 8-dimensional planning problem.
- **Multi-agent distributed training**: Considering that our method decomposes the task into two agents, we compare it with the classical multi-agent RL algorithms MADDPG [31], MAPPO [32], and the heterogeneous-agent RL algorithm HAPPO to evaluate the advantages of the sequential optimization training method for our task. While MADDPG and MAPPO exhibit significant differences in their policy optimization strategies, both leverage the centralized training and decentralized execution (CTDE) architecture, which performs

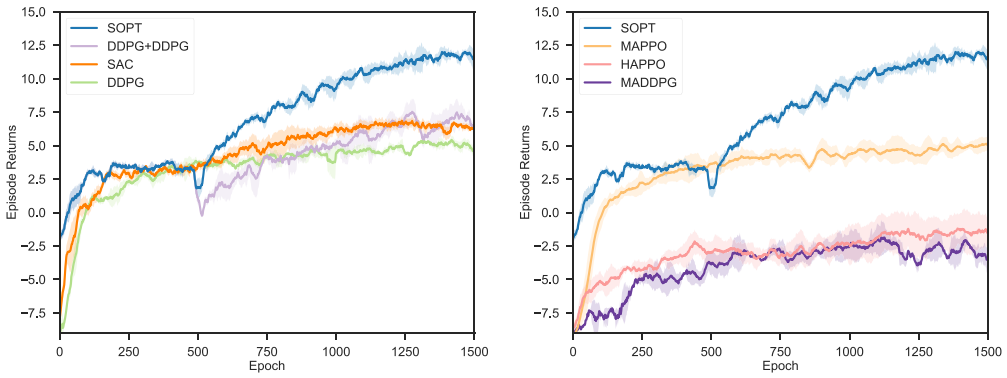


Fig. 7. (a) Performance comparison with single-agent centralized training algorithms and backbone comparison algorithm. The first 500 episode returns for SOPT and DDPG+DDPG are base rewards only. (b) Performance comparison with multi-agent distributed training algorithms. The episode returns for the multi-agent algorithms are the sum of all agent rewards.

Table 1
Test the active visual tracking (AVT) success rate, average base task metrics B_{reori} and B_{stab}. The smaller B_{reori} and B_{stab} are, the higher the base manipulability and stability.

Algorithm	AVT Success rate	B_{reori}	B_{stab}
SOPT	98.91%	2.42661	0.09083
DDPG+DDPG	45.17%	2.42661	0.09083
SAC	39.34%	2.96702	0.27466
DDPG	43.08%	2.59671	0.34880
MAPPO	34.96%	2.45861	0.14617
MADDPG	15.81%	4.96173	0.55060
HAPPO	19.47%	3.62255	0.48502

Table 2
Test the tracking task (TT) success rate and mean error during active visual tracking and target tracking. Tracking task is deemed successful when $e_{\text{p}} \leq 0.015$ m and $e_{\varphi} \leq 0.017$ rad.

Algorithm	TT Success rate	Pos error,m	Ori error,rad
SOPT	94.17%	0.00766	0.00607
DDPG+DDPG	0%	0.21468	0.02251
SAC	0%	0.19551	0.02181
DDPG	0%	0.25301	0.01617
MAPPO	0%	0.17089	0.02576
MADDPG	0%	0.42333	0.29533
HAPPO	0%	0.38562	0.27383

exceptionally well in multi-agent continuous action space tasks. The HAPPO [33] algorithm facilitates effective multi-agent collaboration in general heterogeneous agent settings through coordinated agent updates.

- **Backbone algorithm comparison**: Due to the limitations of non-stationary training, the base is trained using the DDPG algorithm. To evaluate the performance of SAC in the manipulator task, after the base deployed the deterministic policy, we redesigned the backbone algorithm to train the robot using DDPG.

Fig. 7 illustrates a comparison of the training episode return curves for our algorithm and the comparison experiments. The SOPT algorithm demonstrated a steady increase in episodic return throughout the training process, stabilizing after approximately 1250 episodes, which underscores its excellent performance. In contrast, single-agent centralized training algorithms, such as SAC and DDPG, showed slower gains in episodic return, with both algorithms converging to suboptimal solutions. Compared with multi-agent distributed training algorithms, MAPPO exhibited better performance but still did not match the performance of SOPT. The episodic returns of HAPPO and MADDPG increased very slowly and fluctuated significantly. Notably, due to the differing dimensionalities in the action and state spaces of the base and manipulator, all multi-agent algorithms required zero-padding for the lower-dimensional spaces to satisfy learning requirements, including HAPPO. This addition of redundant elements significantly reduced learning efficiency. Although HAPPO was designed for heterogeneous agents, it did not yield the expected results in this context, likely because our task is not an ideal cooperative environment. The coupling between the base and manipulator introduces an element of adversarial interaction. In addition, compared to DDPG+DDPG, the manipulator training with SAC algorithm has a significant advantage.

Table 1 and Table 2 clearly illustrate the policy performance of different algorithms in the testing environment, demonstrating the unique advantages of our proposed method in this task. The SOPT algorithm achieved an AVT success rate of 98.91% and a TT success rate of 94.17%, with average position and orientation errors significantly below the constraints, measuring 0.00766 m and 0.00607 rad, respectively. These results confirm that the SOPT algorithm successfully enables precise tracking of tumbling targets with relative motion over extended periods. The joint task framework successfully connects the active visual tracking and target tracking tasks. In addition, the base has the highest manipulability and stability compared to other algorithms.

It is noteworthy that other algorithms exhibited particularly large position errors, primarily due to the high sensitivity of the end-effector position to changes in the base attitude. During the process of exploring the optimal base attitude, the end-effector learning was subjected to insurmountable disturbances. Furthermore, as position errors decrease, the probability of collision increases, further complicating the learning of precise position control. The coupling between the base and the manipulator, combined with the large action space, resulted in the comparison algorithms failing to achieve the desired performance. This limitation underscores the necessity of our sequential optimization training approach. To illustrate the tracking of a tumbling target in relative motion, a episode was randomly selected for visualization. The snapshot of the process is shown in Fig. 8. The base attitude and angular velocity during tracking are given in Fig. 9. The position and orientation errors of the end-effector, the manipulator joint angles and the joint angular velocities are shown in Figs. 10 and 11. In addition, the maximum offset of the base position during tracking are 0.0508 m.

4.2. Strength testing

To evaluate the strength of our policy, we conducted tests across various scenarios. It is important to clarify that the errors discussed in this section refer to the end-effector pose errors during both the active visual tracking and target tracking phases. Based on the design of the active visual tracking task in the joint task framework, provided that the error remains within the prescribed tolerance, the target will remain within the field of view. Therefore, the success rate of active visual tracking is not presented in this section.

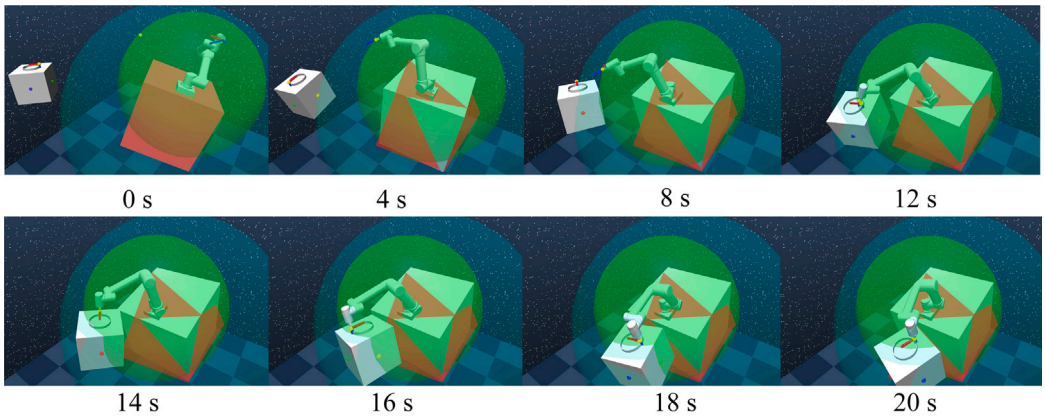


Fig. 8. Video snapshots of the space manipulator tracking the tumbling target.

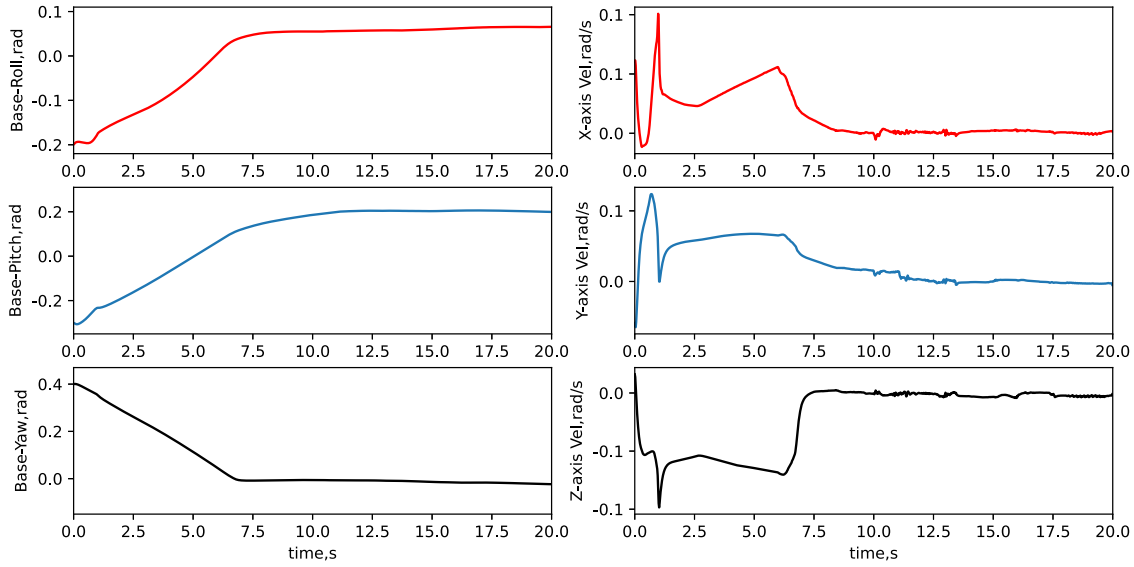


Fig. 9. The attitude and angular velocity of the base.

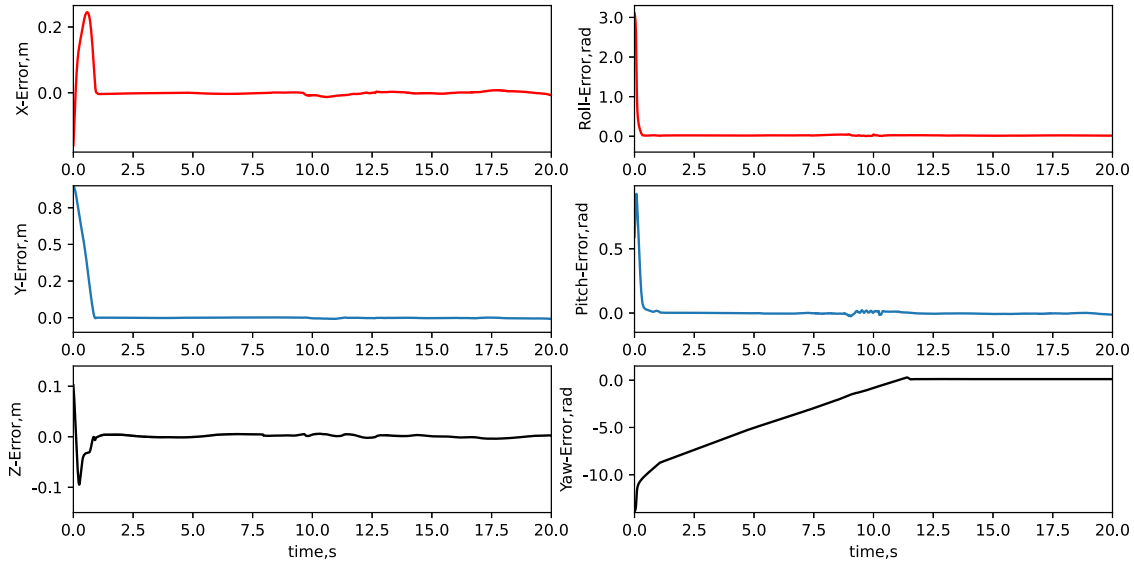


Fig. 10. The position and orientation error between the end-effector and the target.

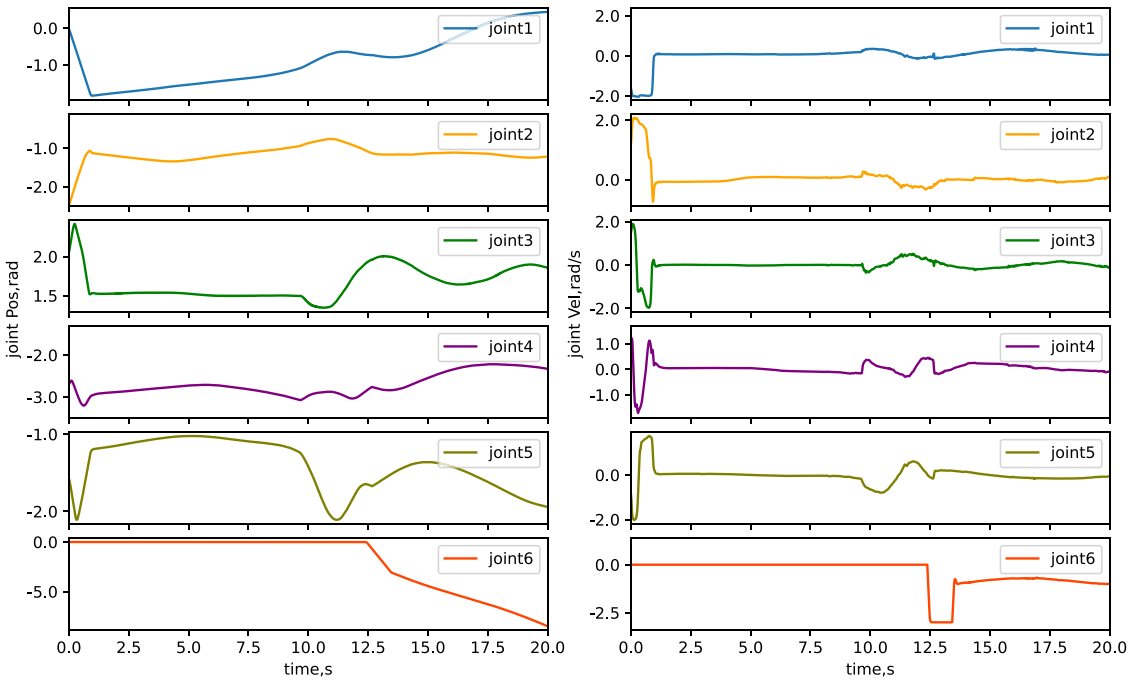


Fig. 11. The manipulator joint angles and joint angular velocities.

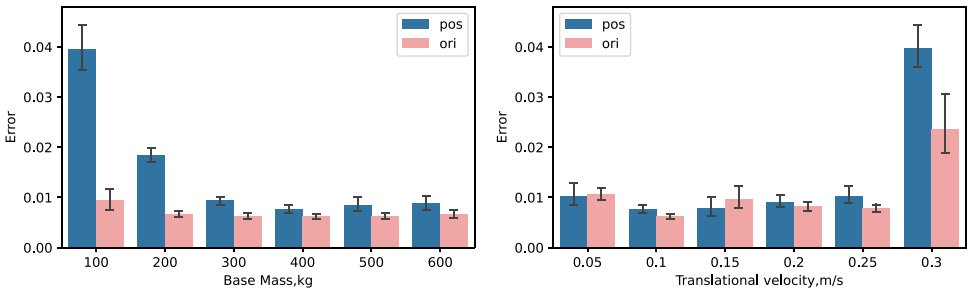


Fig. 12. (a): The position and orientation error under different masses of the base. (b) : The position and orientation error under different target translational velocities.

As previously noted, the position of the end-effector is highly sensitive to changes in the base's posture, and the base mass is a critical factor in influencing the disturbances caused by manipulator movements. Fig. 12(a) clearly illustrates the performance of our policy under different base mass. When the base mass is within the range of 300 kg–600 kg, the error remains within acceptable limits. However, the reduction of the base mass amplifies the effect of the disturbance caused by the manipulator motion. When the base mass decreases by more than 50%, the position error exceeds acceptable thresholds. The significant increase in position tracking error primarily stems from dynamic coupling effects on the base position. The end-effector is far away from the desired position at the initial moment, and it rapidly approaches the desired position within 1 s. The rapid motion of the manipulator results in significant deflection of the base attitude and even cause a shift in the base position. Even though the base policy controls the base to maintain an optimal posture, the increased distance between the base and the target reduces the capture time window, causing the target to exit the dexterous workspace prematurely.

The relative velocity of the NCSO and the space robot is a determining factor in the capture time window. Excessive target translational velocity may result in tracking failure or require the space robot to consume additional fuel to maneuver again to reduce the relative motion. We evaluated the performance of our policy under different translational velocities, as shown in Fig. 12(b). It is worth noting that the translational velocity is random in direction but still guarantees that the target is within the predefined target motion space. Even if the translational velocity reaches 0.25 m/s, our policy still accomplish the task of tracking. Since we strictly limit the base angular velocity to a narrow range based on the actual satellite attitude angular velocity, the space robot is too late to incorporate the target into the dexterous workspace when the translational velocity reaches 0.3 m/s.

In addition, we evaluate the performance of the policy under different tumbling states at target translational velocities of 0.1 m/s and 0.2 m/s, as shown in Fig. 13. The policy trained using our method effectively approximates targets with varying tumbling speeds and translational velocities, enabling continuous and stable tracking. Even when the initial translational velocity is 0.2 m/s and the tumbling velocity reaches [0.1,0.1,2] rad/s, our policy still successfully achieves target tracking. This validates the effectiveness and robustness of our method in planning and tracking NCSOs in relative motion. Since there is always relative motion between space robots and NCSOs in real space environments, our method will provide new possibilities for space robots to capture NCSOs.

A detailed video demonstration is included as supplementary material, providing additional insight into the experimental results presented in this paper.

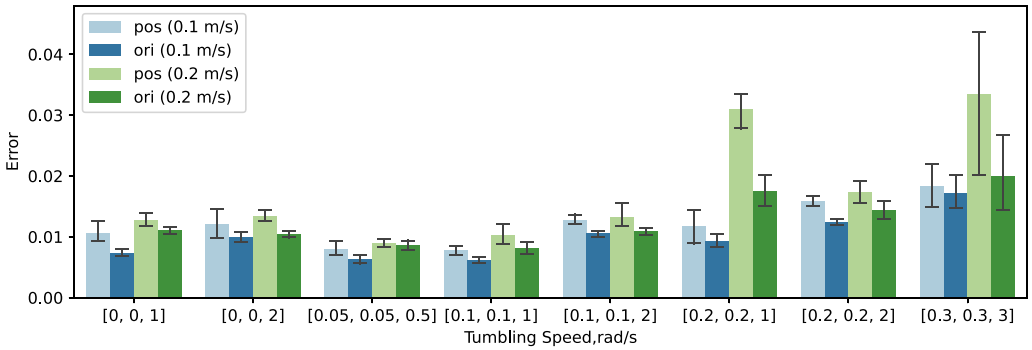


Fig. 13. The position and orientation error of the end-effectors tracking the target with different tumbling states.

Table 3
Parameters for the joint task framework and rewards.

Tasks	Parameters
Base reorientation	$U_{\text{ctw}} = 0.59$ m, $U_{\text{ws}} = 0.25$ rad, $w_{\text{ctw}} = 50$, $w_{\text{ws}} = 50$ and $w_{\text{cost}} = 3.5$
Base stabilization	$U_{t_{\text{reori}}} = 6$ s and $w_{\text{stab}} = 1000$
Active visual tracking	$w_{\text{ap1}} = 0.25$, $w_{\text{ap2}} = 3$, $w_{\text{ap3}} = 4$ and $w_{\text{ap4}} = 0.45$
Target tracking	$\text{t}_{\text{line}} = 150$, $\text{t}_{\text{sl}} = 150$, $w_{\varphi} = 20$
Base reward	$w_{\text{b1}} = 0.25$, $w_{\text{b2}} = 3$, $w_{\text{b3}} = 3$, $w_{\text{b4}} = 5$ and $w_{\text{b5}} = 4$
Manipulator reward	$w_{\text{m1}} = 4$, $w_{\text{m2}} = 2$, $w_{\text{m3}} = 100$, $w_{\text{m4}} = 2$ and $w_{\text{m5}} = 3$

4.3. Discussion

Our research has successfully reduced the learning complexity of free-flying space robots while expanding their operational workspace. This further demonstrates the immense potential of space robots in space applications, offering new possibilities for a range of on-orbit operations, including but not limited to spacecraft maintenance and on-orbit assembly.

While our method has shown promising results in simulations, we acknowledge several limitations that could influence its practical applicability. Our research focuses solely on the active visual tracking and target tracking stages of the capture process. Capturing a relatively moving NCSO imposes strict requirements on other phases of the operation. The tracking task heavily depends on the accuracy for target pose recognition and motion state estimation [34]. Subsequent contact tasks must overcome significant contact forces and ensure the stability of the combined system [35]. Additionally, the structural flexibility of space robots introduces unpredictable vibrations that affect tracking accuracy and overall stability, requiring additional control modules to suppress these disturbances.

In the design of the task scenario, we limit the range and state of the target motion. The NCSO relative positions, translational speeds, and tumbling speeds are more diverse in practice. The huge exploration space requires further improvement of sample learning efficiency. In addition, the backbone algorithms used are off-policy RL algorithms, which may not be directly applicable to real-world systems. Robust RL based on offline data [36] and offline RL [37] may be promising alternatives. Overall, these limitations highlight potential directions for future research and avenues that will enhance the practical applicability of our work.

5. Conclusions

We proposed an RL-based motion planning algorithm for FFSRs to track tumbling targets with relative motion. The sequential optimization method effectively addresses the challenge of slow convergence in high-dimensional planning for FFSRs. The introduced joint tracking task framework seamlessly integrates active visual tracking with target tracking tasks. Simulation results demonstrated that the proposed algorithm can achieve fast convergence and exhibits strong robustness. The operational workspace of the space robot is expanded because the targets in various motion states can be tracked. Although some limitations remain, this method represents a forward-looking solution with promising applications in the future. In our future work, we will explore the contact problem of capturing tumbling targets with relative motion and the challenges in stability of combined spacecraft systems.

CRediT authorship contribution statement

Zhicai Dong: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Formal analysis. **Lu Wang:** Writing – review & editing, Validation, Supervision. **Yiman Zhu:** Writing – review & editing, Software. **Liaoxue Liu:** Writing – review & editing, Validation. **Jian Guo:** Writing – review & editing. **Yu Guo:** Writing – review & editing, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by National Natural Science Foundation of China (62403242), Natural Science Foundation of Jiangsu Province, China (BK20241459), Postgraduate Research & Practice Innovation Program of Jiangsu Province, China (KYCX25_0751) and the Fundamental Research Funds for the Central Universities, China (30924010932).

Appendix A

See Tables 3–5.

Table 4
Parameters of the task environment.

Object	Parameter	Value
	mass [kg]	400
Base	size [m]	$0.94 \times 0.94 \times 0.94$
	initial attitude [rad]	$[-0.2, -0.3, 0.4]^{\mathrm{T}}$
Manipulator	initial joint angle [rad]	$[0, -2.48, 2.04, -2.7, -1.57, 0]^{\mathrm{T}}$
	inertia matrix [$\mathrm{kg \cdot m^2}$]	diag{450 400 350}
Tumbling satellite	size [m]	$0.4 \times 0.4 \times 0.4$
	radius of the satellite ring [m]	0.1
Dexterous workspace	radius [m]	0.79
Reachable workspace	radius [m]	1.418

Table 5
Hyperparameters of the **SOPT**.

Hyperparameters	SOPT
Actor network	(256,256)
Critic network	(256,256)
Learning rate of actor	1.e-3
Learning rate of critic	1.e-3
Optimizer	Adam
Replay buffer size	10^6
Discount(γ)	0.99
Polyak($1-\tau$)	0.995
Batch size	128
Entropy coefficient	0.2

Appendix B. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.actaastro.2025.05.027.

References

[1] V.V. Svotina, Space debris removal – review of technologies and techniques. Rigid coupling between space debris and service spacecraft, J. Space Saf. Eng. 11 (2) (2024) 252–267.

[2] Zhihong Jiang, Xiaolei Cao, Xiao Huang, Hui Li, Marco Ceccarelli, Progress and development trend of space intelligent robot technology, Space: Sci. Technol. 2022 (2022).

[3] Yaqiang Wei, Xiao Yang, Xinlin Bai, Zhigang Xu, Adaptive hybrid Kalman filter for attitude motion parameters estimation of space non-cooperative tumbling target, Aerosp. Sci. Technol. 144 (2024) 108832.

[4] Angel Flores-Abad, Ou Ma, Khanh Pham, Steve Ulrich, A review of space robotics technologies for on-orbit servicing, Prog. Aerosp. Sci. 68 (2014) 1–26.

[5] Ouyang Zhang, Weiran Yao, Desong Du, Chengwei Wu, Jianxing Liu, Ligang Wu, Yu Sun, Trajectory optimization and tracking control of free-flying space robots for capturing non-cooperative tumbling objects, Aerosp. Sci. Technol. 143 (2023) 108718.

[6] Xiutao Gu, Liaoxue Liu, Lu Wang, Zhaobao Yu, Yu Guo, Energy-optimal adaptive artificial potential field method for path planning of free-flying space robots, J. Franklin Inst. 361 (2) (2024) 978–993.

[7] Xiaoyi Wang, Jayantha Katupitiya, Robust control of a dual-arm space robot to capture a non-cooperative target in 3D space, Aerosp. Sci. Technol. 141 (2023) 108538.

[8] Bicheng Cai, Cheng Wei, Chengfei Yue, Yanjun Yu, Xueqin Chen, Yunhai Geng, Artificial potential incorporated adaptive fixed-time sliding mode control for target capture, Adv. Space Res. 72 (4) (2023) 982–996.

[9] Shihao Ni, Weidong Chen, Hehua Ju, Ti Chen, Coordinated trajectory planning of a dual-arm space robot with multiple avoidance constraints, Acta Astronaut. 195 (2022) 379–391.

[10] Lei Yan, Wenfu Xu, Zhonghua Hu, Bin Liang, Multi-objective configuration optimization for coordinated capture of dual-arm space robot, Acta Astronaut. 167 (2020) 189–200.

[11] Matteo D'Ambrosio, Lorenzo Capra, Andrea Brandonisio, Stefano Silvestrini, Michèle Lavagna, Redundant space manipulator autonomous guidance for in-orbit servicing via deep reinforcement learning, Aerospace 11 (5) (2024) 341.

[12] Zhicai Dong, Yiman Zhu, Lu Wang, Yifei Wu, Yu Guo, Motion planning of free-floating space robots for tracking tumbling targets by two-axis matching via reinforcement learning, Aerosp. Sci. Technol. 155 (2024) 109540.

[13] Kirk Hovell, Steve Ulrich, Laboratory experimentation of spacecraft robotic capture using deep-reinforcement-learning–based guidance, J. Guid. Control Dyn. 45 (11) (2022) 2138–2146.

[14] Ahmad Al Ali, Jian-Feng Shi, Zheng H. Zhu, Path planning of 6-DOF free-floating space robotic manipulators using reinforcement learning, Acta Astronaut. 224 (2024) 367–378.

[15] Yinkang Li, Danyi Li, Wenshan Zhu, Jun Sun, Xiaolong Zhang, Shuang Li, Constrained motion planning of 7-DOF space manipulator via deep reinforcement learning combined with artificial potential field, Aerospace 9 (3) (2022).

[16] Yuxue Cao, Shengjie Wang, Xiang Zheng, Wenke Ma, Xinru Xie, Lei Liu, Reinforcement learning with prior policy guidance for motion planning of dual-arm free-floating space robot, Aerosp. Sci. Technol. 136 (2023) 108098.

[17] Yun-Hua Wu, Zhi-Cheng Yu, Chao-Yong Li, Meng-Jie He, Bing Hua, Zhi-Ming Chen, Reinforcement learning in dual-arm trajectory planning for a free-floating space robot, Aerosp. Sci. Technol. 98 (2020) 105657.

[18] Shengjie Wang, Yuxue Cao, Xiang Zheng, Tao Zhang, A learning system for motion planning of free-float dual-arm space manipulator towards non-cooperative object, Aerosp. Sci. Technol. 131 (2022) 107980.

[19] Shengjie Wang, Xiang Zheng, Yuxue Cao, Tao Zhang, A multi-target trajectory planning of a 6-DoF free-floating space robot via reinforcement learning, in: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2021, pp. 3724–3730.

[20] Dezhi ZHANG, Guocai YANG, Yongjun SUN, Junhong JI, Minghe JIN, Hong LIU, Two-phase visual servoing for capturing tumbling non-cooperative satellites with a space manipulator, Chin. J. Aeronaut. 37 (12) (2024) 560–573.

[21] Mengqing Hong, Xiutao Gu, Lingling Xia, Yu Guo, Fast fixed-time extended state observer based command filtering backstepping control of free-flying flexible-joint space robots, J. Franklin Inst. 361 (17) (2024) 107183.

[22] Guocai Yang, Junhong Ji, Yechao Liu, Songhao Piao, A novel visual servo method for space manipulators by planning jerk, Adv. Space Res. 71 (6) (2023) 2591–2608.

[23] Kim Mathiassen, Jørgen Enger Fjellin, Kyrre Glette, Per Kristian Hol, Ole Jakob Elle, An ultrasound robotic system using the commercial robot UR5, Front. Robot. AI 3 (2016) 1.

[24] Yoji Umetani, Kazuya Yoshida, Workspace and manipulability analysis of space manipulator, Trans. Soc. Instrum. Control Eng. 26 (2) (1990) 188–195.

[25] Dong Zhou, Guanghui Sun, Wenxiao Lei, Ligang Wu, Space noncooperative object active tracking with deep reinforcement learning, IEEE Trans. Aerosp. Electron. Syst. 58 (6) (2022) 4902–4916.

[26] Wenxiao Lei, Hao Fu, Guanghui Sun, Active object tracking of free floating space manipulators based on deep reinforcement learning, Adv. Space Res. 70 (11) (2022) 3506–3519.

[27] Shengjie Wang, Yuxue Cao, Xiang Zheng, Tao Zhang, Collision-free trajectory planning for a 6-DoF free-floating space robot via hierarchical decoupling optimization, IEEE Robot. Autom. Lett. 7 (2) (2022) 4953–4960.

[28] Yingce Zhao, Guanghao Zhang, Zhengyu Xing, Jianxun Li, Hierarchical reinforcement learning adversarial algorithm against opponent with fixed offensive strategy, J. Shanghai Jiaotong Univ. (Sci.) 29 (3) (2024) 471–479.

[29] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, Martin Riedmiller, Deterministic policy gradient algorithms, in: International Conference on Machine Learning, Pmlr, 2014, pp. 387–395.

[30] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, Sergey Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: International Conference on Machine Learning, PMLR, 2018, pp. 1861–1870.

[31] Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, Igor Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, Adv. Neural Inf. Process. Syst. 30 (2017).

[32] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, Yi Wu, The surprising effectiveness of ppo in cooperative multi-agent games, Adv. Neural Inf. Process. Syst. 35 (2022) 24611–24624.

[33] Yifan Zhong, Jakub Grudzien Kuba, Xidong Feng, Siyi Hu, Jiaming Ji, Yaodong Yang, Heterogeneous-agent reinforcement learning, J. Mach. Learn. Res. 25 (1–67) (2024) 1.

[34] Tae Ha Park, Simone D'Amico, Robust multi-task learning and online refinement for spacecraft pose estimation across domain gap, Adv. Space Res. 73 (11) (2024) 5726–5740, Recent Advances in Satellite Constellations and Formation Flying.
[35] Yinkang Li, Bin Song, Qiufan Yuan, Shuang Li, Time-optimal coordinated detumbling and pose adjusting control of a non-cooperative target with model uncertainties, Acta Astronaut. 220 (2024) 204–217.
[36] Aditya M. Deshpande, Ali A. Minai, Manish Kumar, Robust deep reinforcement learning for quadcopter control, IFAC- Pap. 54 (20) (2021) 90–95, Modeling, Estimation and Control Conference MECC 2021.
[37] Jeyeon Eo, Dongsu Lee, Minhae Kwon, The impact of dataset on offline reinforcement learning performance in UAV-based emergency network recovery tasks, IEEE Commun. Lett. 28 (5) (2024) 1058–1061.